

บทที่ 3

การป้องกันการเกิด Buffer Overflow

มีวิธีการหลายอย่างถูกใช้เพื่อป้องกันหรือตรวจสอบการเกิด buffer overflow ซึ่งมีทั้งข้อดีข้อเสียแตกต่างกันออกไป ซึ่งมีทั้งการป้องกันในระดับ source code, compile time โดยสามารถแบ่งออกเป็นหัวข้อย่อยๆ ได้ดังต่อไปนี้

3.1 Static Code analysis

เป็นการป้องกันการเกิด buffer overflow โดยการวิเคราะห์ source code ว่า เสี่ยงต่อการเกิด buffer overflow หรือไม่ โดยดูจากการใช้ฟังก์ชันที่ไม่ถูกต้อง การเขียนโปรแกรมโดยไม่ได้ตรวจสอบขอบเขตของตัวแปร รวมทั้งการใช้ฟังก์ชันที่เสี่ยงต่อการทำให้เกิด buffer overflow

3.1.1 ฟังก์ชันที่เสี่ยงต่อการโจมตีด้วยวิธี buffer overflow

ฟังก์ชัน: strcpy

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์ขณะที่ทำสำเนาไปยังเป้าหมาย

การแก้ไข: เปลี่ยนไปใช้ ฟังก์ชัน strncpy หรือ strlcpy

ฟังก์ชัน: lstrcpy, wcscpy, _tscpy, _mbscopy

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์ขณะที่ทำสำเนาไปยังเป้าหมาย

การแก้ไข: ใช้ฟังก์ชันที่ทำการหยุดสำเนาเมื่อสิ้นสุดบัฟเฟอร์

ฟังก์ชัน: memcpy, CopyMemory, bcopy

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์ขณะที่ทำสำเนาไปยังเป้าหมาย

การแก้ไข: ทำให้ตัวแปรที่จะนำมารับค่ามีขนาดพอเพียงสำหรับการรับค่า

ฟังก์ชัน: strcat

ความปลอดภัย: ไม่ได้ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์ขณะที่ทำการเชื่อมต่อความไปยังเป้าหมาย

การแก้ไข: เปลี่ยนไปใช้ฟังก์ชัน strncat หรือ strlcat แทน

ฟังก์ชัน: `lstrcat, wcsat, _tscat, _mbcat`

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์ขณะที่ทำการเชื่อมต่อความไปยังเป้าหมาย

ฟังก์ชัน: `strncpy`

ความปลอดภัย: ยากต่อการใช้อย่างถูกต้อง ไม่ได้ทำการปิดด้วย \0 เสมอไป หรือตรวจสอบตัวชี้ที่ไม่ถูกต้อง

ฟังก์ชัน : `lstrepyn, wcsncpy, _tcsncpy, _mbsnbcpy`

ความปลอดภัย: ยากต่อการใช้อย่างถูกต้อง ไม่ได้ทำการปิดด้วย \0 เสมอไป หรือตรวจสอบตัวชี้ที่ไม่ถูกต้อง

ฟังก์ชัน: `strncat`

ความปลอดภัย: ยากต่อการใช้งานที่ไม่ถูกต้อง (เช่นการคำนวณค่าสูงสุดที่ตัวแปรสามารถรับได้)
การแก้ไข: ใช้ฟังก์ชัน `strlcat` หรือ คำนวณค่าสูงสุดที่ตัวแปรรับได้ให้ถูกต้อง

ฟังก์ชัน: `lstrcatn, wcsncat, _tcsncat, _mbsnbc`

ความปลอดภัย: ยากต่อการใช้งานที่ไม่ถูกต้อง (เช่นการคำนวณค่าสูงสุดที่ตัวแปรสามารถรับได้)
การแก้ไข: ใช้ฟังก์ชัน `strlcat` หรือ คำนวณค่าสูงสุดที่ตัวแปรรับได้ให้ถูกต้อง

ฟังก์ชัน: `strncpy, strcadd`

ความปลอดภัย: เกิดการล้นของบัฟเฟอร์ถ้าบัฟเฟอร์ไม่ใหญ่พอ
การแก้ไข: ทำให้แน่ใจว่าบัฟเฟอร์มีขนาดใหญ่เพียงพอ

ฟังก์ชัน: `gets, _getts`

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์
การแก้ไข: ใช้ `fgets` แทน

ฟังก์ชัน: `sprintf, vsprintf, swprintf, vswprintf, _stprintf, _vstprintf`

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์
การแก้ไข: ใช้ฟังก์ชัน `snprintf` หรือ `vsnprintf` แทน

ฟังก์ชัน: `scanf, vscanf, wscanf, _tscanf`

ความปลอดภัย: รับตัวแปรโดยไม่กำหนดขอบเขตของตัวแปร สามารถทำให้เกิดการล้นของบัฟเฟอร์ได้

การแก้ไข: กำหนดขอบเขตของตัวแปรให้แน่นอนหรือใช้ฟังก์ชันในการรับค่าฟังก์ชันอื่น

ฟังก์ชัน: `fscanf, sscanf, vsscanf, vfscanf, _ftscanf`

ความปลอดภัย: รับตัวแปรโดยไม่กำหนดขอบเขตของตัวแปร สามารถทำให้เกิดการล้นของบัฟเฟอร์ได้

การแก้ไข: กำหนดขอบเขตของตัวแปรให้แน่นอนหรือใช้ฟังก์ชันในการรับค่าฟังก์ชันอื่น

ฟังก์ชัน: `strlen, wcslen, _tcslen, _mbslen`

ความปลอดภัย: ไม่สามารถจัดการกับ string ที่ไม่มี \0 ปิดท้าย

ฟังก์ชัน: `MultiByteToWideChar`

ความปลอดภัย: ต้องการความยาวเป็นตัวอักษร ไม่ใช่เป็นไบต์

ฟังก์ชัน: `streadd, strecpy`

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์

การแก้ไข: ทำให้แน่ใจว่าตัวแปรที่จะรับมีขนาดเป็น 4 เท่าของตัวแปรที่ส่งมา เพื่อจะได้มีที่ว่างสำหรับการขยาย

ฟังก์ชัน: `strtrns`

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์

การแก้ไข: ทำให้แน่ใจว่าตัวแปรที่รับข้อมูลมีขนาดใหญ่กว่าตัวแปรที่ส่งข้อมูล

ฟังก์ชัน: `realpath`

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์

การแก้ไข: ทำให้แน่ใจว่าตัวแปรที่รับข้อมูลมีขนาดใหญ่กว่าขนาดของ MAXPATHLEN และทำการป้องกันปัญหาจากการ implement โดยตัวแปรที่รับเข้าไปในฟังก์ชันต้องไม่มีค่ามากไปกว่า MAXPATHLEN

ฟังก์ชัน: `getopt, getopt_long`

ความปลอดภัย: การ implement แบบเก่าไม่ได้ป้องกันการล้นของบัฟเฟอร์

การแก้ไข: ตรวจสอบการ implementation ในการติดตั้งหรือจำกัดขนาดของข้อความที่จะใส่ลงไป

ฟังก์ชัน: `getpass`

ความปลอดภัย: การ implement บางครั้งไม่ได้ป้องกันการล้นของบัฟเฟอร์

ฟังก์ชัน: `getwd`

ความปลอดภัย: ไม่ได้ตรวจสอบการเกิดการล้นของบัฟเฟอร์

การแก้ไข: ใช้ฟังก์ชัน `getcwd`

ฟังก์ชัน: `getchar, fgetc, getc, read, _getc`

ความปลอดภัย: ตรวจสอบขอบเขตของบัฟเฟอร์ถ้าใช้ในรูป

ฟังก์ชัน: `getenv, curl_getenv`

ความปลอดภัย: ตัวแปรแวดล้อมไม่สามารถเชื่อถือได้ซึ่งผู้บุกรุกสามารถตั้งค่าได้

การแก้ไข: ตรวจสอบตัวแปรแวดล้อมให้ดีก่อนใช้

ฟังก์ชัน: `g_get_home_dir`

ความปลอดภัย: ฟังก์ชันนี้เหมือนกับคำสั่ง `getenv("HOME")` ซึ่งจะส่งค่าที่เชื่อถือไม่ได้กลับมาถ้าผู้บุกรุกสามารถแก้ไขได้

การแก้ไข: ตรวจสอบตัวแปรแวดล้อมให้ดีก่อนใช้

ฟังก์ชัน: `g_get_tmp_dir`

ความปลอดภัย: ฟังก์ชันนี้เหมือนกับคำสั่ง `getenv("TMP")` ซึ่งจะส่งค่าที่เชื่อถือไม่ได้กลับมาถ้าผู้บุกรุกสามารถแก้ไขได้

การแก้ไข: ตรวจสอบตัวแปรแวดล้อมให้ดีก่อนใช้

3.1.2 การเขียนโปรแกรมโดยไม่ตรวจสอบขอบเขตของตัวแปร

ในการเขียนโปรแกรมใดโปรแกรมหนึ่ง ผู้เขียนโปรแกรมต้องคอยระวังการประกาศตัวแปร โดยเฉพาะตัวแปรชนิด อาร์เรย์ ยกตัวอย่างเช่น

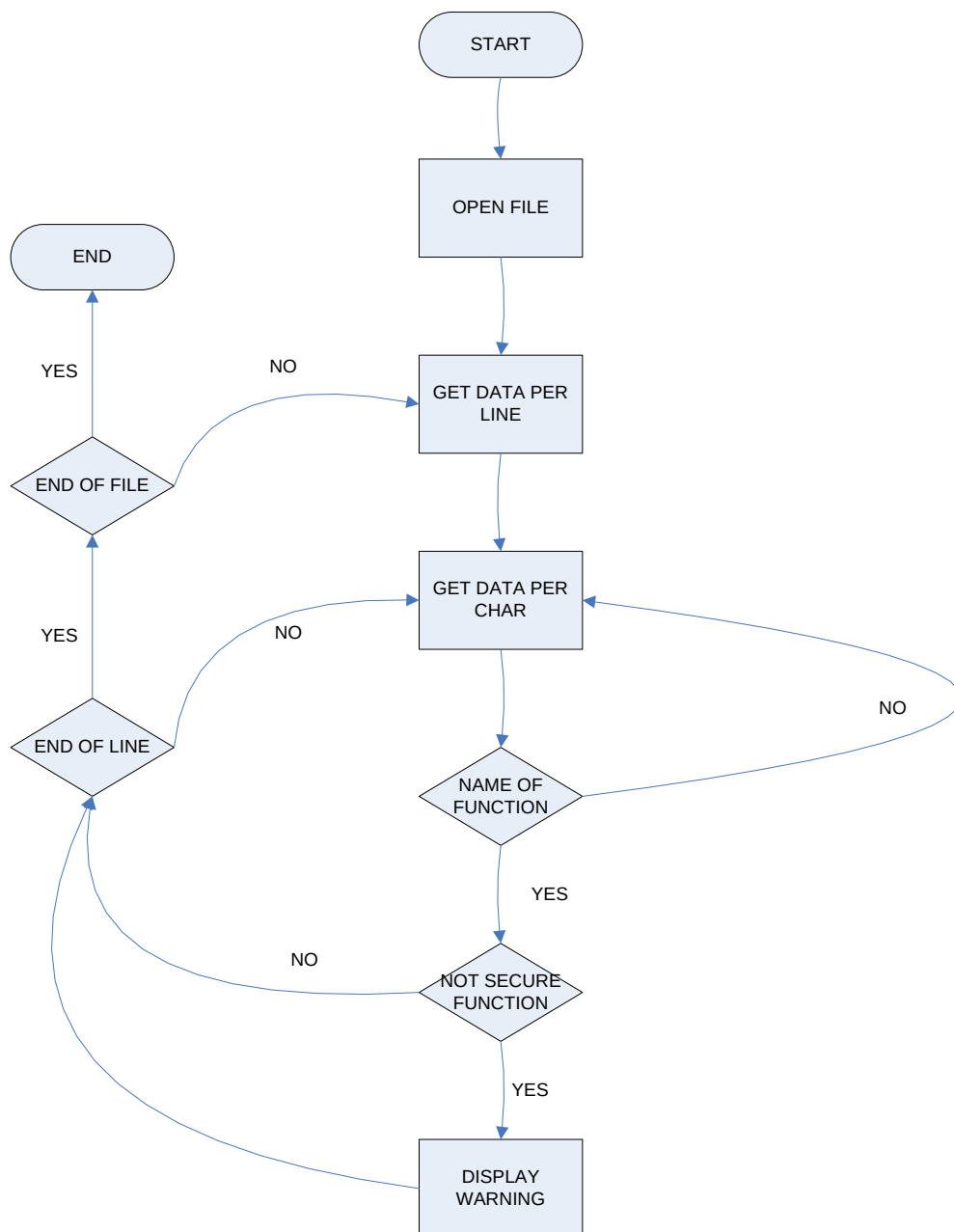
```
char buf[10];

buf[10] = A;
```

ซึ่งเมื่อทำการ compile โปรแกรมนี้ ย่อมไม่มีสิ่งผิดปกติเกิดขึ้น แต่เมื่อใดก็ตามที่ทำการเรียกโปรแกรมนี้ขึ้นมาทำงาน ก็จะทำให้เกิด segmentation fault เกิดขึ้น เนื่องจาก ตัวแปร buf จะมีตัวแปรที่ใส่ได้แค่ 0 – 9 ถ้าทำการให้ค่าไปที่อาร์เรย์ตัวที่ 10 ย่อมก่อให้เกิด segmentation fault ขึ้นแน่นอน ซึ่งสามารถเป็นช่องโหว่ในการโจมตีได้

3.1.3 Lancelot Scan

Lancelot Scan เป็นโปรแกรมที่ได้พัฒนาขึ้นเพื่อการศึกษา โดยเป็นโปรแกรมป้องกันการเกิด Buffer Overflow แบบ Static Code analysis โดยมีการทำงานตาม Flow chart ดังนี้



รูปที่ 3.1 โฟลว์ชาร์ตของโปรแกรม Lancelot Scan

3.1.3.1 ขั้นตอนการทำงาน

- โปรแกรมทำการเปิด file source code ที่ต้องการวิเคราะห์
- อ่านข้อมูลจาก source code ออกมาทีละบรรทัด แล้วทำการแยก string ออกมาเป็นอักขระ แล้วทำการวิเคราะห์ว่าเป็น ฟังก์ชันหรือไม่ ถ้าไม่ก็จะทำการค้นหาต่อไปจนจบบรรทัด ถ้าเป็นฟังก์ชันก็จะส่งไปตรวจสอบดูว่า เป็นฟังก์ชันที่เสี่ยงต่อการเกิด buffer overflow หรือไม่

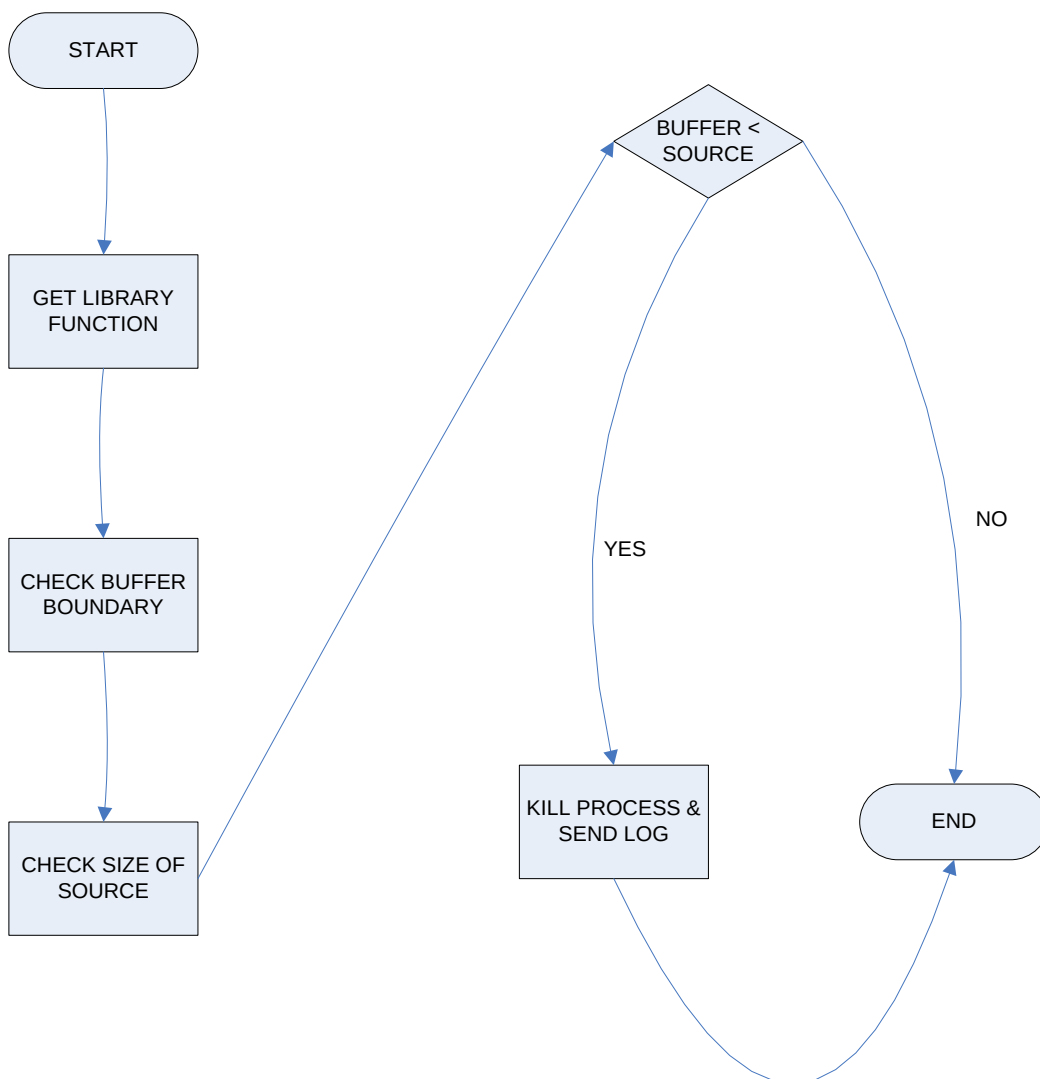
- ทำการตรวจสอบว่าเป็นฟังก์ชันที่เสี่ยงต่อการเกิด buffer overflow หรือไม่ ถ้าใช่ ก็ทำการแสดงผลออกทางหน้าจอแล้วค้นหาฟังก์ชันต่อไป ถ้าไม่ใช่ก็ข้ามไปหาฟังก์ชันอื่น จนหมดบรรทัด
- ทำการอ่านบรรทัดถัดไปของ source code จนหมด จึงจบการทำงาน

3.2 Stack-smashing protection

เป็นการป้องกันการเกิด buffer overflow โดยการตรวจสอบไปที่ stack ว่าจะต้องไม่มีการเปลี่ยนแปลงเมื่อฟังก์ชันคืนค่ากลับไป ซึ่งสามารถทำได้หลายวิธีด้วยกัน

3.2.1 Lancelot Guard

Lancelot Guard เป็นโปรแกรมที่พัฒนาเพื่อการศึกษาการป้องกัน buffer overflow แบบ Stack-smashing protection โดยมีหลักการทำงานเป็น flow chart ดังนี้



รูปที่ 3.2 โฟลว์ชาร์ตของโปรแกรม Lancelot Guard

3.2.1.1 ขั้นตอนการทำงาน

- Stack Guard จะทำหน้าที่ดักฟังก์ชันที่โปรแกรมเรียกใช้ เช่น strcpy() เป็นต้น จากนั้นทำการคำนวณความกว้างของ buffer ว่าสามารถรองรับได้ขนาดเท่าไร โดยดูจากตำแหน่งของตัวแปรที่ถูกกระทำ กับ pointer ESP
- ทำการเทียบดูว่า ฟังก์ชันเขียนข้อมูลลงไปมากกว่าขอบเขตที่ buffer มีอยู่หรือไม่ ถ้ามากกว่า ให้ทำการ kill process นั้นไปแล้วส่งค่า log ไปยัง /var/log/secure แต่ถ้าไม่ก็ให้ process นั้นทำงานต่อไป
- ฟังก์ชันที่ lancelot guard สามารถดักได้นั้นคือ

```
char *(*strcpy_t) (char *dest, const char *src, size_t n);
void *(*memcpy_t) (void *dest, const void *src, size_t n);
char *(*strcpy_t) (char *dest, const char *src);
wchar_t *(*wcscpy_t) (wchar_t *dest, const wchar_t *src);
char *(*stpcpy_t) (char *dest, const char *src);
wchar_t *(*wcpcpy_t) (wchar_t *dest, const wchar_t *src);
char *(*strcat_t) (char *dest, const char *src);
char *(*strncat_t) (char *dest, const char *src, size_t n);
wchar_t *(*wscat_t) (wchar_t *dest, const wchar_t *src);
char *(*getwd_t) (char *buf);
```

3.2.1.2 ตัวอย่างการทำงานของ lancelot guard

```
char *stpcpy(char *dest, const char *src)
fp = __builtin_frame_address(2) // ตำแหน่งเริ่มต้นของ stack

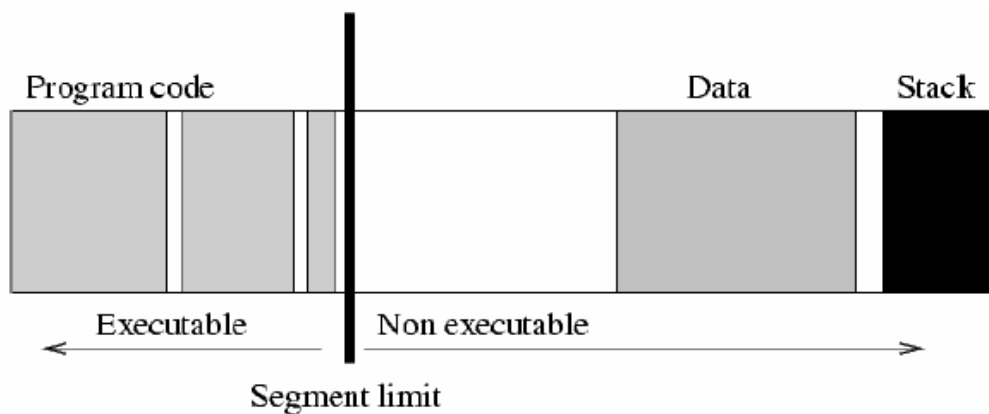
bufsize = fp - dest; //หาขนาดของบัฟเฟอร์

if ((len = strlen(src, bufsize)) == bufsize){ //ตรวจสอบว่าเลข
บัฟเฟอร์หรือไม่

    send_log("stpcpy()"); // ส่งค่า log
    activated("Overflow caused by stpcpy()"); // kill process
}
```

3.3 การป้องกัน Executable space

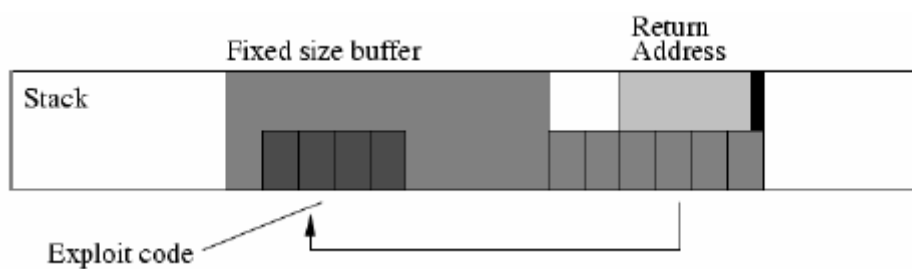
การป้องกัน executable space เป็นการป้องกันการเกิด buffer overflow โดยการทำให้กระบวนการทำงานของมัน ไม่สามารถทำงานได้ ซึ่งทำได้ด้วยการสุ่มตำแหน่ง address (ASLR) แล้วทำให้ส่วนของ memory ไม่สามารถเขียนหรือเรียกทำงาน (execute) ได้ ซึ่งทำให้ shellcode ที่ส่งเข้าไปไม่สามารถทำงานได้นั่นเอง ตัวอย่างของวิธีการนี้ได้แก่ PaX และ Exec-Shield



รูปที่ 3.3 รูปแสดงการป้องกัน Executeable space

3.4 การสุ่มตำแหน่งพื้นที่ของ address

ในการโจมตีด้วยวิธี Stack overflow จะเห็นว่าผู้โจมตีจะพยายามเขียนทับค่า return address บน stack เพื่อชี้ไปยัง buffer



รูปที่ 3.4 รูปแสดงการสุ่มตำแหน่งพื้นที่ของ address

จากรูปจะเห็นว่าจะต้องรู้ตำแหน่ง address คร่าวๆ ของ buffer บน stack สำหรับการเขียนทับไปยัง return address เพื่อชี้กลับมาในตำแหน่งที่ shell code อยู่ ดังนั้นจึงมีการสุ่มตำแหน่งเพื่อเป็นการยากในการค้นหาตำแหน่งที่แน่นอนของ buffer โดยตัวอย่างของโปรแกรมที่ป้องกันการเกิด buffer overflow นี้ได้แก่ exec-shield-randomizes ซึ่งจะทำการสุ่ม

- Stack ของโปรแกรม
- ตำแหน่งของ share Library
- จุดเริ่มต้นของ Heap ในโปรแกรม

3.5 Intrusion-detection software

การใช้ Intrusion-detection software (IDS) เป็นการป้องกันการเกิด buffer overflow โดยอาศัยการจดจำรูปแบบของการทำโจมตี เช่น shell code ที่นิยมใช้ในการโจมตี รวมไปถึงการใช้ NOP Sled ซึ่งเป็นการใช้คำสั่ง No Operation ติดต่อกันมากจนผิดปกติ เป็นต้น